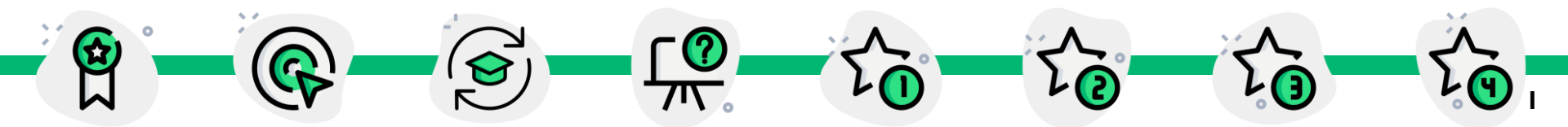


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.03.16

2주차



기본 자료구조 (배열, 포인터와 구조체)



02-1 배열이란?

◆ 자료구조 정의하기

- 자료구조란 데이터 단위와 데이터 자체 사이의 물리적 또는 논리적인 관계를 말함.

◆ 데이터 단위란?

- 데이터를 구성하는 하나의 덩어리이다.
- 자료구조는 쉽게 말해 자료를 효율적으로 사용할 수 있도록 컴퓨터에 저장하는 방법을 말한다.

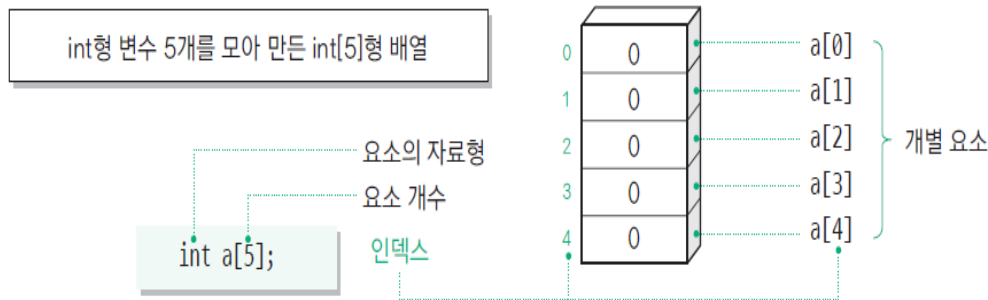


02-1 배열이란?

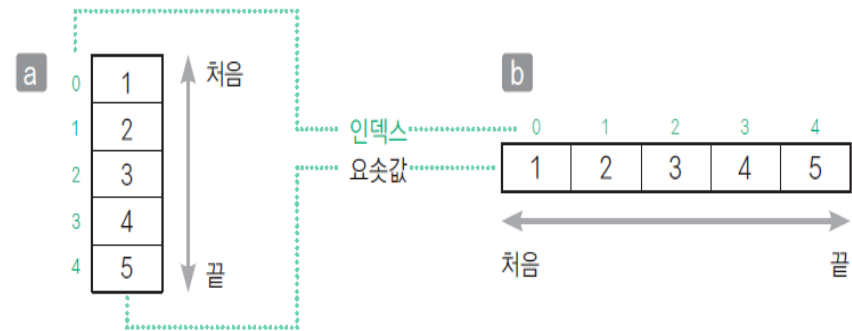
◆ 배열이란?(1)

■ 배열 array

- ✓ 배열은 **같은 자료형**의 변수로 이루어진 요소(element)가 모여 직선 모양으로 줄지어 있는 자료구조이다.
- ✓ 박스 안의 숫자나 문자가 **요소값**이고, 박스의 왼쪽 또는 위쪽에 쓰인 작은 숫자가 **인덱스 값**이다.
- ✓ 요소를 세로로 정렬 시 인덱스가 작은 값을 위쪽에 오도록 한다.
- ✓ 요소를 가로로 정렬 시 인덱스가 작은 값을 왼쪽에 오도록 한다.



[그림 2-2] 배열



[그림 2-4] 배열의 표기



02-1 배열이란?

◆ 배열이란?(2) – 실습 2-1

- 자료형 배열 이름[요소 개수];
 - ✓ 배열의 선언 시 → `int a[5];`
 - ✓ 요소 개수는 상수만 사용할 수 있음
- 배열 이름[인덱스]
 - ✓ 첫 번째 배열 요소의 인덱스는 0이다.
 - ✓ 요소가 n 개인 배열의 요소는 `a[0], a[1], ..., a[n - 1]`이다.



02-1 배열이란?

Chapt02_예제 2-1/intary.c

```
// int[5]형 배열(자료형이 int형이고 요소가 5개)에 값을 입력해 출력
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

#define N 5          // 배열의 요소 개수
int main()
{
    int a[N];         // 배열의 선언
    for (int i = 0; i < N; i++) { // 각 요소에 값을 입력
        printf("a[%d] : ", i);
        scanf("%d", &a[i]);
    }
    puts("각 요소의 값");
    for (int i = 0; i < N; i++) { // 각 요소의 값을 출력
        printf("a[%d] = %d\n", i, a[i]);
    }

    return 0;
}
```



02-1 배열이란?

◆ 배열 다루기(3) – 예제 2-2

- 배열의 요솟값을 초기화하며 배열을 선언하는 예제
- 각 요소를 처음부터 순서대로 쉼표(,)로 구분하여 줄지어 놓고 {}로 둘러싸으로써 배열 a의 요소 a[0], a[1], a[2], a[3], a[4]를 순서대로 1, 2, 3, 4, 5로 초기화



02-1 배열이란?

Chapt02_예제 2-2/intary_init.c

```
// int형 배열을 초기화하고 출력
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};
    int na = sizeof(a) / sizeof(a[0]); // 요소의 개수
    printf("배열 a의 요소 개수는 %d입니다.\n", na);

    for (int i = 0; i < na; i++)
        printf("a[%d] = %d\n", i, a[i]);

    return 0;
}
```

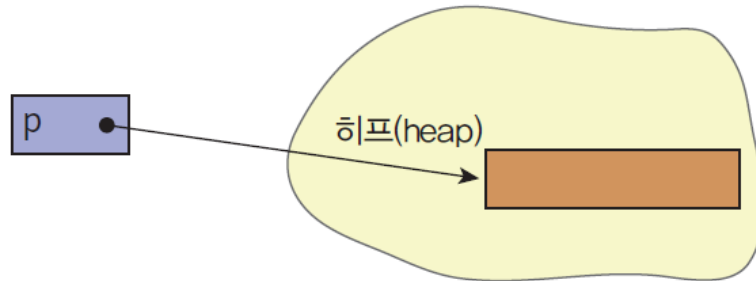



02-1 배열이란? - 동적 메모리 할당

◆ 메모리 할당과 동적 객체 생성하기 - (1)

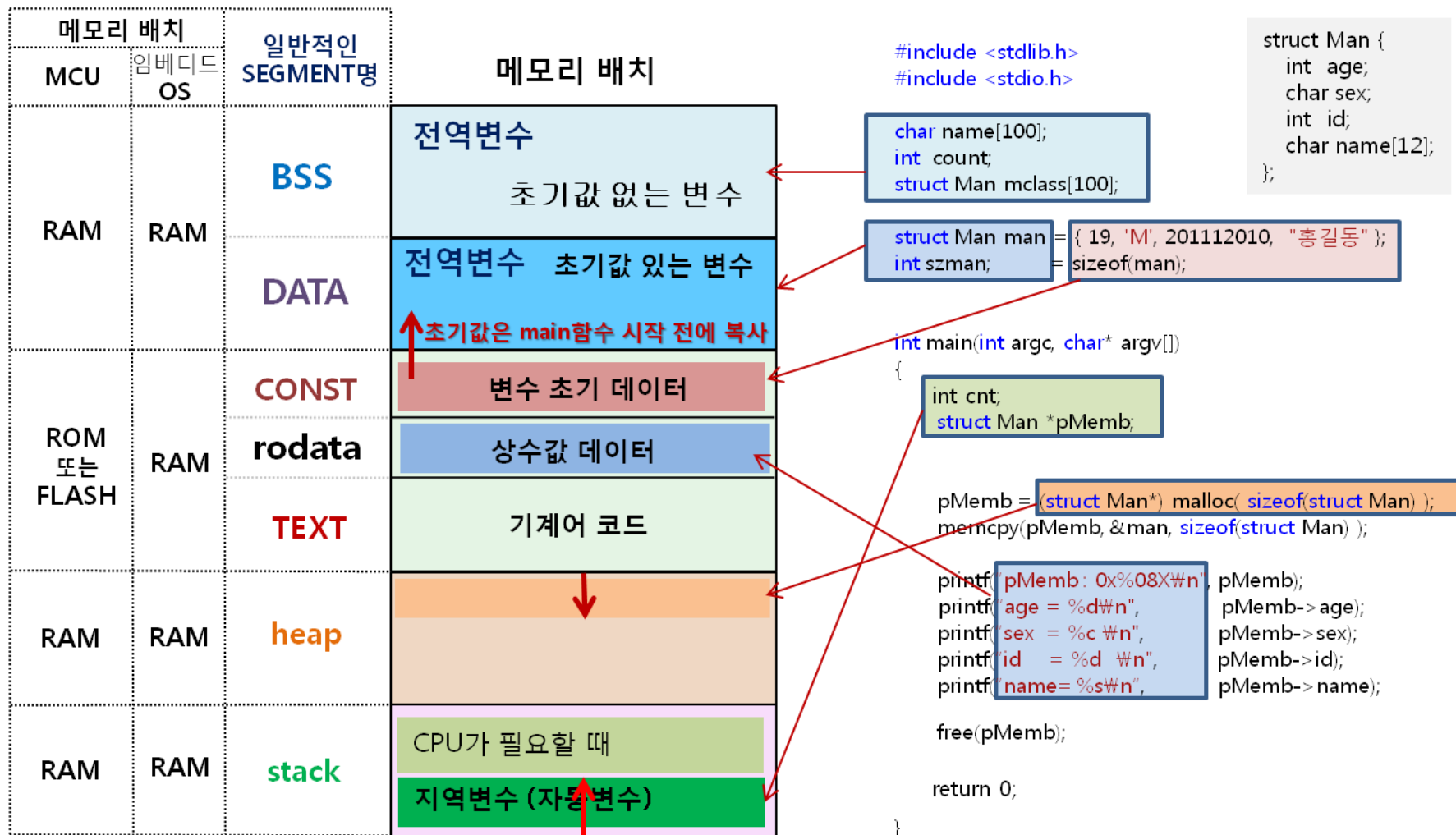
■ 메모리 할당

- ✓ 동적 메모리 할당에서 calloc, malloc 함수는 힙(heap)이라는 메모리에 동적으로 기억 장소를 확보한다.
- ✓ 프로그램의 실행 도중에 메모리를 할당 받는 것
- ✓ 필요한 만큼만 할당을 받고 또 필요한 때에 사용하고 반납
- ✓ 메모리를 매우 효율적으로 사용가능





02-1 배열이란? - 동적 메모리 할당





02-1 배열이란? - 동적 메모리 할당

◆ 메모리 할당과 동적 객체 생성하기 - (2)

■ 메모리 해제

- ✓ 확보한 메모리가 불필요하면 그 공간을 해제해야 하는데 이를 위해 제공되는 함수가 free 함수
- ✓ free 함수를 사용하면 프로그램을 실행하는 도중에도 원하는 시점에 변수를 생성하거나 제거할 수 있음

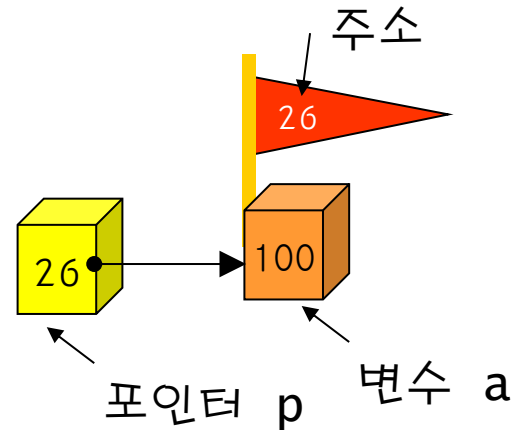
free 함수	
헤더	#include <stdlib.h>
형식	void free(void *ptr);
해설	ptr이 가리키는 메모리를 해제하여 이후에 다시 할당할 수 있도록 합니다. ptr이 NULL 포인터인 경우 아무 것도 하지 않습니다. 이때 ptr로 전달된 실인수(actual argument)가 calloc 함수, malloc 함수, realloc 함수에 의해 반환된 포인터가 아니거나 영역이 free 함수, realloc 함수를 호출하여 이미 해제된 영역이면 아무것도 하지 않습니다.
반환값	없음



02-1 배열이란? - 동적 메모리 할당

- 포인터: 다른 변수의 주소를 가지고 있는 변수

```
int a=100;  
int *p;  
p = &a;
```

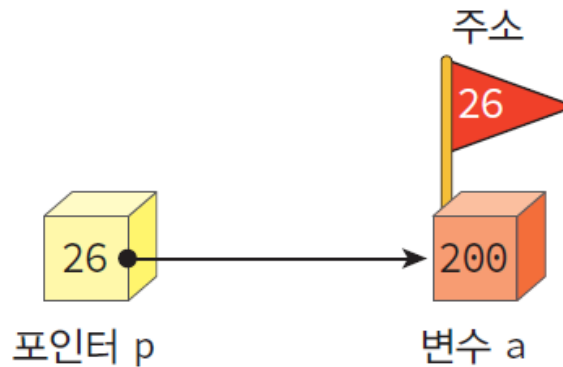




02-1 배열이란? - 동적 메모리 할당

- 포인터가 가리키는 내용의 변경: * 연산자 사용

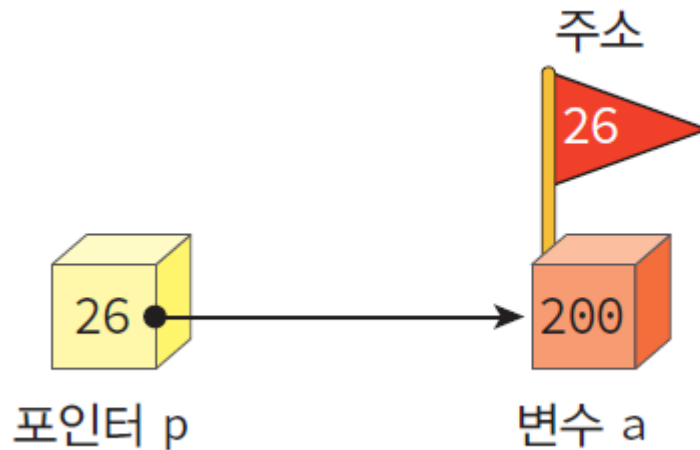
```
*p= 200;
```





02-1 배열이란? - 동적 메모리 할당

- & 연산자: 변수의 주소를 추출
- * 연산자: 포인터가 가리키는 곳의 내용을 추출





02-1 배열이란? - 동적 메모리 할당

◆ 메모리 할당과 동적 객체 생성하기 - (3) 예제 2-3

- 하나의 int형 변수를 calloc 함수로 생성해 정수값을 대입, 출력한 후 free 함수로 해제하는 프로그램
- **Type *x = calloc(1, sizeof(Type)); free(x);**
 - ✓ Type형의 단일 객체 *x의 생성과 해제
 - ✓ 1 * sizeof(int) 바이트 크기의 메모리를 힙에 할당
 - ✓ 확보한 메모리 영역은 *x로 접근할 수 있음



02-1 배열이란? - 동적 메모리 할당

Chapt02_예제 2-3/int_dynamic.c

```
// int형 객체를 동적으로 생성하고 해제
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>

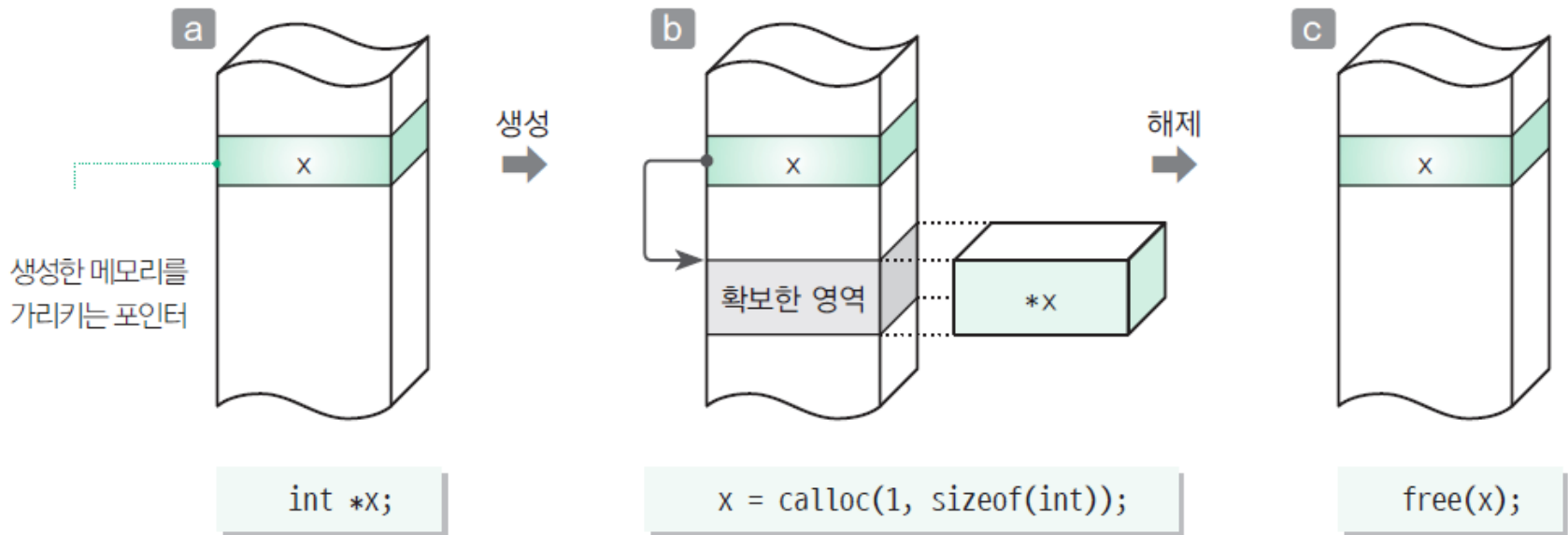
int main(void){
    int* x = calloc(1, sizeof(int)); // int형 포인터에 메모리 할당
    if (x == NULL)
        puts("메모리 할당에 실패했습니다.");
    else {
        *x = 57; // x는 주소값을 저장하고 있고, *x = 57;은 그 주소가 가리키는 메모리 공간에
        // 57을 저장하는 의미
        printf("*x = %d\n", *x);
        free(x); // int형 포인터에 할당한 메모리 해제
    }
    return 0;
}
```




02-1 배열이란? - 동적 메모리 할당

◆ 배열 다루기(4) - 예제 2-3

- `Type *x = calloc(1, sizeof(Type)); free(x);`



[그림 2-5] 객체의 동적 생성과 해제



02-1 배열이란? - 동적 메모리 할당

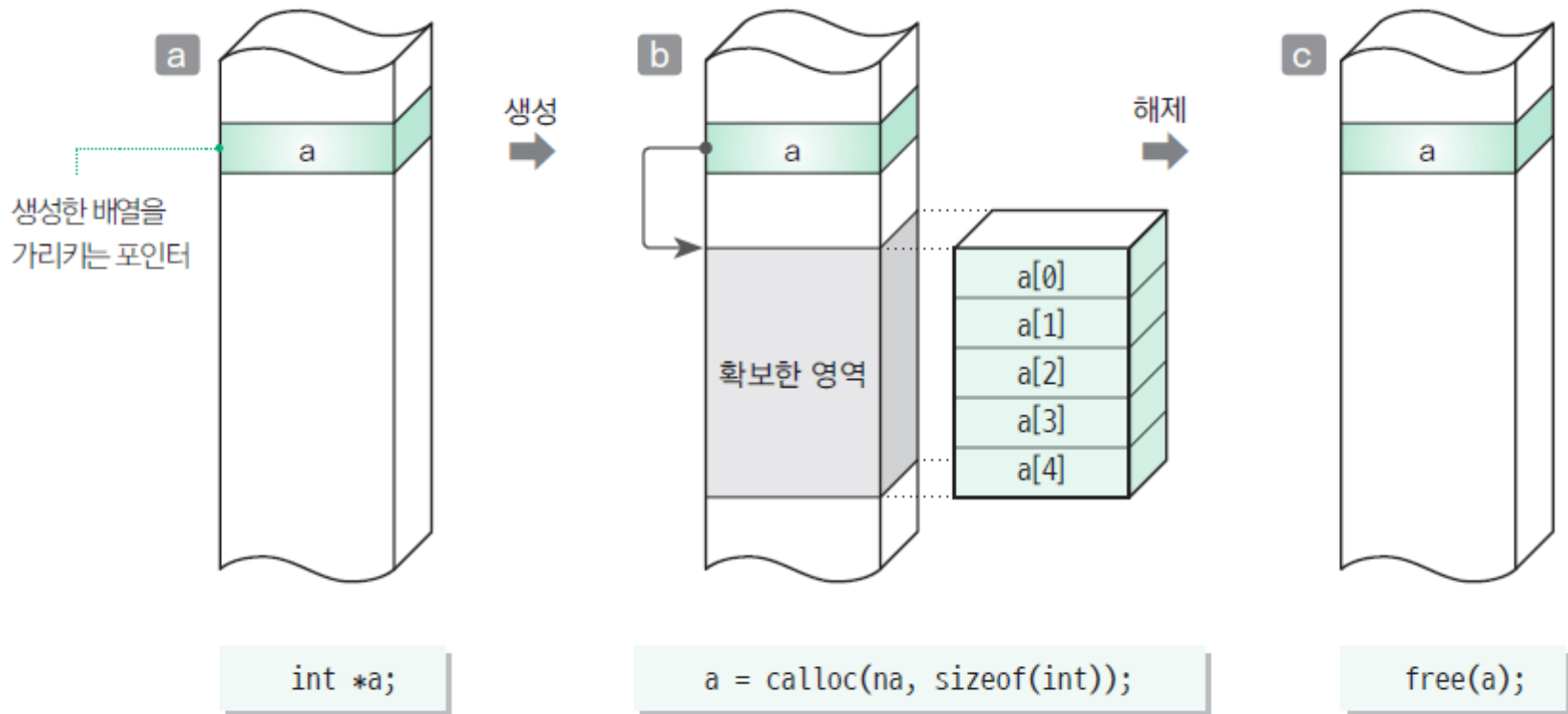
◆ 배열을 동적으로 생성하기 - 예제 2-4

- 자료형이 int형인 배열 객체를 동적으로 생성하는 프로그램
- `Type *a = calloc(n, sizeof(Type)); free(a);`
 - ✓ 요소 개수 $na * \text{sizeof(int)}$ 바이트 크기의 메모리를 힙에 할당
 - ✓ 확보한 메모리의 요소는 인덱스 식 $a[0], a[1], a[2] \dots$ 등으로 접근할 수 있음



02-1 배열이란? - 동적 메모리 할당

◆ 배열을 동적으로 생성하기 - 예제 2-4



[그림 2-6] 배열의 동적 생성과 해제(요소 개수 $na = 5$)



02-1 배열이란? - 동적 메모리 할당

Chapt02_예제 2-4/intary_dynamic.c

```
// int형 배열을 동적으로 생성하고 해제
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int na; // 배열 a의 요소 개수
    printf("요소 개수 : ");
    scanf("%d", &na);

    int* a = calloc(na, sizeof(int)); // 요소 개수가
    na인 int형 배열을 생성
```

```
if (a == NULL)
    puts("메모리 확보에 실패했습니다.");
else {
    printf("%d개의 정수를 입력하세요.\n", na);
    for (int i = 0; i < na; i++) {
        printf("a[%d] : ", i);
        scanf("%d", &a[i]);
    }
    printf("각 요솟값은 아래와 같습니다.\n");
    for (int i = 0; i < na; i++)
        printf("a[%d] = %d\n", i, a[i]);
    free(a); // 요소 개수가 na인 int형
    배열을 해제
}
return 0;
}
```



02-1 배열이란? – 배열 요소의 최대값 구하기

◆ 배열 요소의 최대값 구하기 – (1)

■ 배열 요소의 최대값 구하기

- ✓ $a[0], a[1], \dots, a[n-1]$ 의 최대값을 구하는 알고리즘
- ✓ ● 안의 값은 인덱스를 나타냄
- ✓ 1 → $a[0]$ 의 값을 확인 후 $a[0]$ 의 값을 max로 저장
- ✓ 2 → for 문에서는 $a[1]$ 부터 $a[n-1]$ 까지 차례로 살펴보아 max보다 큰 값일 때 max 값과 교환

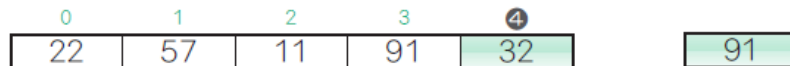
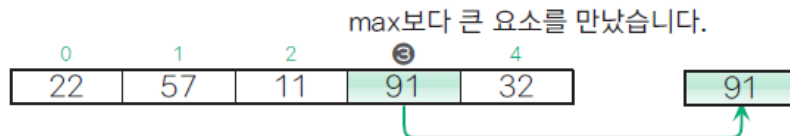
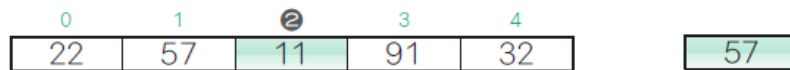
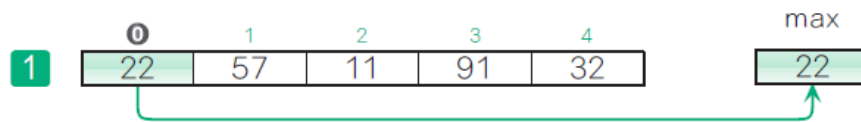
■ 주사

- ✓ 배열의 요소를 하나씩 차례로 살펴보는 과정



02-1 배열이란? – 배열 요소의 최대값 구하기

```
max = a[0];  
for(int i = 1; i < n; i++)  
    if(a[i] > max) max = a[i];
```



[그림 2-8] 배열 요소의 최대값을 구하는 과정의 한 예



02-1 배열이란? – 배열 요소의 최대값 구하기

◆ 배열 요소의 최대값 구하기 – (2)

■ 실습 2-5 – (1)

- ✓ 배열에서 가장 큰 값을 찾는 기능을 maxof 함수로 구현
- ✓ maxof 함수는 배열 a에서 최대값을 찾아 반환
- ✓ height 배열은 사람의 키를 저장하는 용도로 사용
- ✓ 사용자로부터 인원 수를 입력받아 변수 number에 저장하고, 크기가 number인 int형 배열 height를 동적으로 생성
- ✓ 배열의 각 요소에 사용자 입력 값을 저장한 후, height와 number를 maxof 함수에 전달하여 최대값을 계산
- ✓ maxof 함수는 배열 요소 중 가장 큰 값을 찾아 반환
- ✓ 프로그램 종료 전에 free 함수를 사용하여 동적으로 할당한 메모리를 해제



02-1 배열이란? – 배열 요소의 최대값 구하기

Chapt02_실습 2-5/ary_max.c

// 배열 요소의 최대값을 구하고 출력(요솿값
입력)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/*-- 요소 개수가 n인 배열 a의 요소의 최대값 --*/
```

```
int maxof(const int a[], int n)
```

```
{
```

```
    int max = a[0];        // 최대값
```

```
    for (int i = 1; i < n; i++)
```

```
        if (a[i] > max) max = a[i];
```

```
    return max;
```

```
}
```

```
int main(void)
```

```
{
```

```
    int number;           // 인원 = 배열 height의 요소 개수
```

```
    printf("사람 수: ");
```

```
    scanf("%d", &number);
```

```
    int* height = calloc(number, sizeof(int)); // 요소 개수
```

number개인 배열을 생성

```
    printf("%d명의 키를 입력하세요.\n", number);
```

```
    for (int i = 0; i < number; i++) {
```

```
        printf("height[%d] : ", i);
```

```
        scanf("%d", &height[i]);
```

```
    }
```

```
    printf("최대값은 %d입니다.\n", maxof(height, number));
```

```
    free(height);         // 배열 height를 해제
```

```
    return 0;
```

```
}
```




02-1 배열이란? – 다차원 배열 만들기

◆ 다차원 배열 만들기

- **다차원 배열** multidimensional array

- ✓ 배열을 요소로 하는 배열
- ✓ 배열을 자료형으로 하면 2차원 배열이고, 2차원 배열을 자료형으로 하면 3차원 배열

- **1차원 배열**

- ✓ 지금까지 배운 ‘단일 자료형을 가지는 배열’

- **2차원 배열의 도출**

- ✓ **a** int형 ... int 자료형
- ✓ **b** int[3]형 ... int를 자료형으로 하는 단일 요소가 3개인 배열
- ✓ **c** int[4][3]형 ... int를 자료형으로 하는 단일 요소가 3개인 배열을 자료형으로 하는 요소 개수가 4개인 배열



02-1 배열이란? – 다차원 배열 만들기

a 단일한 int형

int형



3개를 모음

b 1차원 배열(int[3]형)

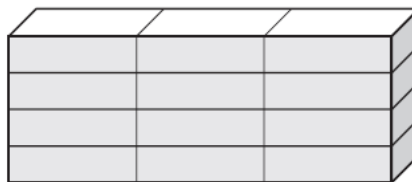
자료형은 int형, 요소 개수는 3



4개를 모음

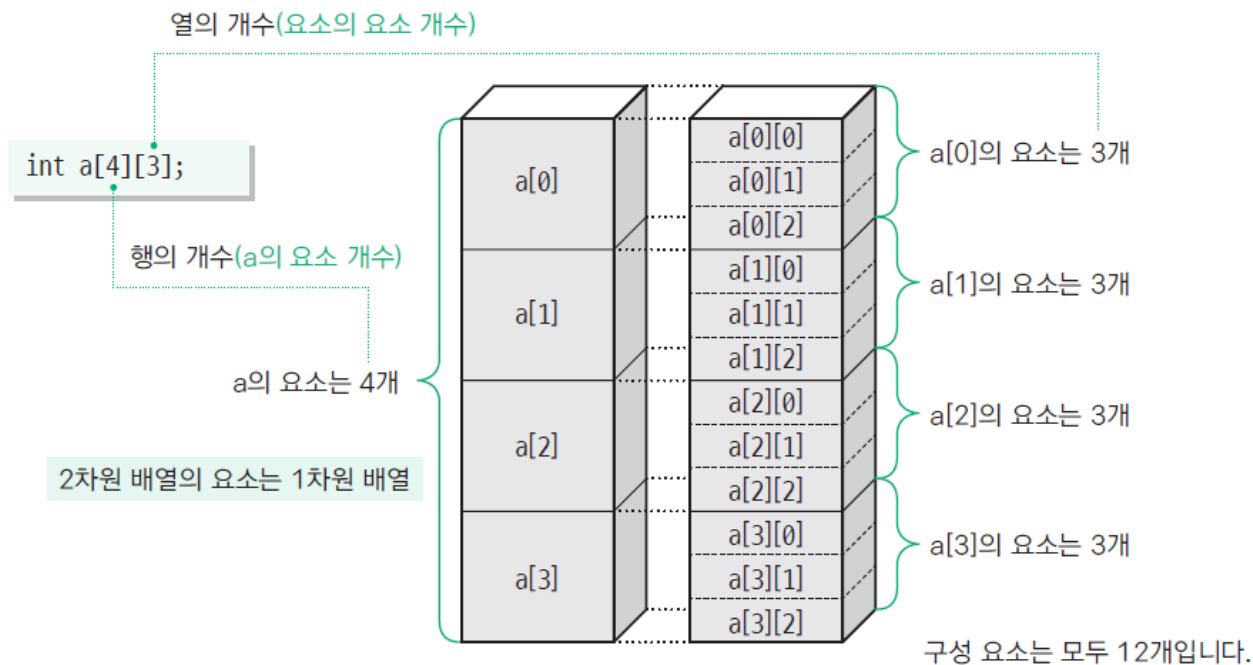
c 2차원 배열(int[4][3]형)

자료형은 int[3]형, 요소 개수는 4



4행 3열의 2차원 배열

[그림 2-16] 2차원 배열의 도출



[그림 2-17] 4행 3열의 2차원 배열



02-1 배열이란? – 다차원 배열 만들기

◆ 날짜를 계산하는 프로그램 만들기-(1)

■ 예제 2-12 – (1)

- ✓ 평년의 2월 날 수는 28일이지만 윤년의 2월 날 수는 29일로, 날 수가 평년인지 윤년인지에 따라 달라짐
- ✓ `dayof_year` 함수는 각 달의 날 수를 저장하는 2차원 배열 `mdays`를 사용하여 2월을 특별히 다루지 않고 날 수를 계산
- ✓ `is leap` 함수는 매개변수 `year`로 받은 연도가 윤년이면 1을, 평년이면 0을 반환하는 함수
- ✓ `y`년 `i`월의 날 수는 `mdays[is leap(y)][i - 1]`로 구할 수 있음

월 - 1의 값을 인덱스로 합니다.

	0	1	2	3	4	5	6	7	8	9	10	11
평년 0	31	28	31	30	31	30	31	31	30	31	30	31
윤년 1	31	29	31	30	31	30	31	31	30	31	30	31

[그림 2-18] 각 달의 날 수를 저장한 2차원 배열



02-1 배열이란? – 다차원 배열 만들기

◆ 날짜를 계산하는 프로그램 만들기-(2)

■ 예제 2-12 – (1)

- ✓ 년, 월, 일의 3개 값이 주어지면 이를 이용해 해당하는 연도의 지난 날 수를 구하는 프로그램

■ `int isleap(int year)` 함수

- ✓ `year % 4 == 0 && year % 100 != 0`: 연도가 4로 나누어 떨어지면서 동시에 100으로 나누어 떨어지지 않는 경우.
- ✓ `year % 400 == 0`: 연도가 400으로 나누어 떨어지는 경우.
- ✓ 이 두 조건 중 하나라도 참이면 윤년으로 간주

- 4의 배수인 해는 윤년이다.
- 100의 배수인 해는 윤년이 아니다.
- 400의 배수인 해는 윤년이다.



02-1 배열이란? – 다차원 배열 만들기

Chapt02_예제 2-12/dayof_year.c

```
// 한 해의 지난 날 수를 구하여 출력
#include <stdio.h>
/*- 각 달의 날 수 -*/
int mdays[][12] = {
    {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
};
/*--- year년이 윤년인가? ---*/
int isleap(int year)
{
    return year % 4 == 0 && year % 100 != 0 ||
    year % 400 == 0;
}
/*--- y년 m월 d일의 그 해 지난 날 수를 구함 ---*/
int dayof_year(int y, int m, int d)
{
    int days = d;    // 날 수
    for (int i = 1; i < m; i++)
        days += mdays[isleap(y)][i - 1];
    return days;
}
```

```
int main(void)
{
    int retry;    // 다시?
    do {
        int year, month, day;    // 년, 월, 일
        printf("년: "); scanf("%d", &year);
        printf("월: "); scanf("%d", &month);
        printf("일: "); scanf("%d", &day);
        printf("%d년의 %d일째입니다.\n", year,
        dayof_year(year, month)); → 오류가 있음
        printf("다시 할까요?(1 ... 예 / 0 ... 아니오): ");
        scanf("%d", &retry);
    } while (retry == 1);

    return 0;
}
```



02-2 구조체란?

◆ 구조체 살펴보기-(1)

■ 구조체 structure

✓ 임의의 자료형의 요소를 조합하여 다시 만든 자료구조

■ 구조체형과 멤버의 접근

✓ **1** 구조체 선언

- 구조체에 붙는 이름인 xyz를 구조체 태그(structure tag)라고 함
- 구조체를 구성하는 요소를 구조체 멤버(structure member)라고 함

✓ **2** 구조체형을 갖는 객체 정의

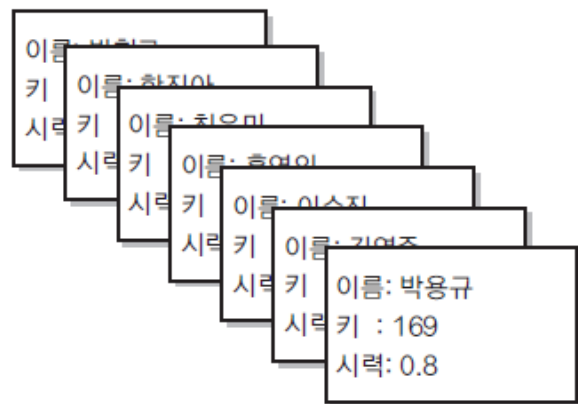
- struct xyz형을 갖는 객체 a를 정의
- 구조체의 객체 안 멤버는 . 연산자를 사용하여 접근

✓ **3** 포인터가 객체를 가리키도록 선언

- p가 구조체형 객체에 대한 포인터일 때 p가 가리키는 객체의 멤버 x에 접근하는 형식은 -> 연산자를 사용함



02-2 구조체란?



0	박현규	162	0.3
1	함진아	173	0.7
2	최윤미	175	2.0
3	홍연의	171	1.5
4	이수진	168	0.4
5	김영준	174	1.2
6	박용규	169	0.8

[그림 2-20] 이름, 키, 시력을 한 세트로 만든 '카드' 배열

구조체의 내용

```

/*--- 구조체 xyz ---*/
struct xyz {
    int    x;  // int형 멤버
    long   y;  // long형 멤버
    double z;  // double형 멤버
};

```

```

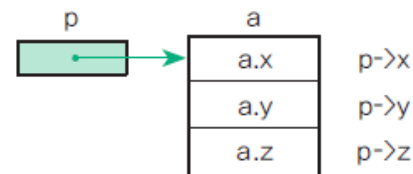
/*--- struct xyz형 a의 정의 ---*/
struct xyz a;

```

```

/*--- a를 가리키는 포인터 ---*/
struct xyz *p = &a;

```



[그림 2-21] 구조체형과 멤버의 접근



02-2 구조체란?

◆ 구조체 배열로 구현하기-(1)

■ 예제 2-13 - (1)

- ✓ 앞부분에서 선언하고 정의하는 구조체의 이름은 PhysCheck
- ✓ 이 구조체는 이름(문자열), 키(int형), 시력(double형)을 가짐
- ✓ ave_height 함수는 신체검사 데이터의 배열을 받아 키의 평균을 실수(double)로 구하는 함수
- ✓ dist_vision 함수는 시력 분포를 구하는 함수
 - 분포를 저장하는 곳은 세 번째 인수 dist
 - 시력 분포는 0.1 단위로 구함


```

1 // 신체검사 데이터용 구조체 배열
2 #include <stdio.h>
3 #define VMAX    21    // 시력의 최댓값 2.1 × 10
4
5 /*--- 신체검사 데이터형 ---*/
6 typedef struct {
7     char    name[20];    // 이름
8     int     height;      // 키
9     double  vision;      // 시력
10 } PhysCheck;
11
12 /*--- 키의 평균값 ---*/
13 double ave_height(const PhysCheck dat[], int n)
14 {
15     double sum = 0;
16     for (int i = 0; i < n; i++)
17         sum += dat[i].height;
18     return sum / n;
19 }
20
21 /*--- 시력 분포 ---*/
22 void dist_vision(const PhysCheck dat[], int n, int dist[])
23 {
24     for (int i = 0; i < VMAX; i++)
25         dist[i] = 0;
26     for (int i = 0; i < n; i++)
27         if (dat[i].vision >= 0.0 && dat[i].vision <= VMAX / 10.0)
28             dist[(int)(dat[i].vision * 10)]++;
29 }

```

실행 결과

■ □ ■ 신체검사표 ■ □ ■

이름 키 시력

박현규	162	0.3
함진아	173	0.7
최윤미	175	2.0
홍연의	171	1.5
이수진	168	0.4
김영준	174	1.2
박용규	169	0.8

평균 키: 170.3 cm

시력 분포

0.0 ~: 0 명

0.1 ~: 0 명

0.2 ~: 0 명

0.3 ~: 1 명

0.4 ~: 1 명

0.5 ~: 0 명

... 생략 ...

```

31 int main(void)
32 {
33     PhysCheck x[] = {
34         {"박현규", 162, 0.3},
35         {"함진아", 173, 0.7},
36         {"최윤미", 175, 2.0},
37         {"홍연의", 171, 1.5},
38         {"이수진", 168, 0.4},
39         {"김영준", 174, 1.2},
40         {"박용규", 169, 0.8},
41     };
42     int nx = sizeof(x) / sizeof(x[0]);           // 사람 수
43     int vdist[VMAX];                             // 시력 분포
44     puts("■■■ 신체검사표 ■■■");
45     puts("   이름           키   시력 ");
46     puts("-----");
47     for (int i = 0; i < nx; i++)
48         printf("%-18.18s%4d%5.1f\n", x[i].name, x[i].height, x[i].vision);
49     printf("\n평균 키: %5.1fcm\n", ave_height(x, nx));
50     dist_vision(x, nx, vdist);                   // 시력 분포
51     printf("\n시력 분포\n");
52     for (int i = 0; i < VMAX; i++)
53         printf("%3.1f ~ : %2d명\n", i / 10.0, vdist[i]);
54
55     return 0;
56 }

```

퀴즈